Sistemas Digitales — Semana 2

Del AVR al ESP32

En AVR teníamos registros DDR, PORT y PIN. En el ESP32 la idea sigue siendo configurar entradas, salidas y leer estados, pero los registros tienen otros nombres y dispone de más funciones para conectar periféricos.

- ATmega328P: 8 bits, trabaja normalmente a 5 V, 16 MHz. Tiene pines asociados a puertos fijos.
- ESP32: 32 bits, trabaja con lógica de 3,3 V. Puede usar Wi-Fi y Bluetooth; tiene una matriz GPIO para conectar señales a distintos pines.

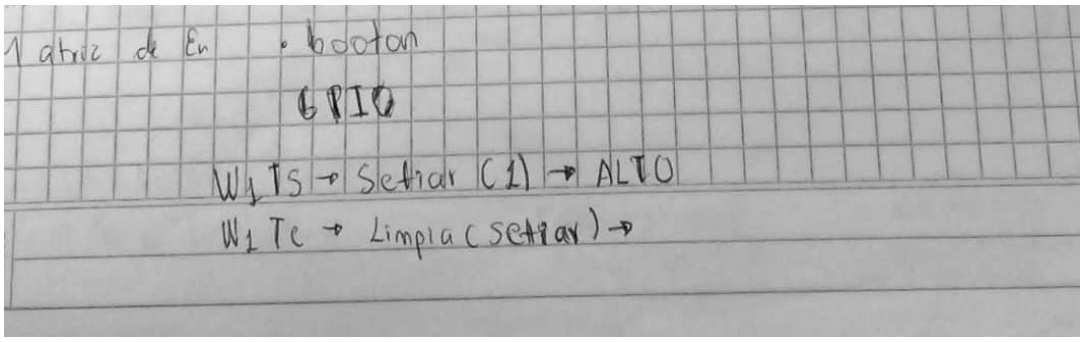
En el cuaderno anotamos ESP32-WROOM-32, alimentación, USB y regulador. La placa DevKit suele recibir 5 V por USB y el regulador entrega 3,3 V al chip. No aplicar 5 V directamente a los GPIO.

Pines, registros y GPIO

Un pin es una conexión física del chip. Los registros son posiciones de memoria que permiten configurar y controlar el hardware. No son lo mismo que el pin.

- En el ESP32 hay registros para poner bits en 1 o limpiarlos sin modificar los demás. W1TS (write one to set) pone en 1 y W1TC (write one to clear) limpia a 0.
- La configuración de pines también depende de si el GPIO admite entrada, salida o tiene funciones especiales de arranque (strapping).
- Los GPIO 34 a 39 del ESP32 clásico son solo entradas y no tienen pull-up/pull-down internos.

Ejemplo: para encender una salida se escribe el bit correspondiente en el registro de set. Para apagarla se escribe en el de clear; se evita cambiar accidentalmente otros bits.



Cómo arranca un programa

Al encender la placa, el chip inicia el arranque y ejecuta el programa guardado en Flash. En Arduino primero se ejecuta setup() una vez y luego loop() se repite continuamente. Es el llamado superlazo.

Sistemas Digitales — Semana 2

Variables y tipos de datos

En los apuntes aparece uint8_t / int8_t. El tipo de dato indica cuántos bits se utilizan y cómo se interpreta el número. Una variable ocupa un espacio en memoria RAM.

- uint8_t: entero sin signo de 8 bits, valores de 0 a 255.
- int8_t: entero con signo de 8 bits, valores de -128 a 127.
- uint32_t: entero sin signo de 32 bits, útil para trabajar con tiempos.

Un mismo conjunto de bits puede interpretarse distinto según el tipo. Por ejemplo 11001100 equivale a 204 en uint8_t, pero a -52 en int8_t.

Hay que recordar que los tamaños de algunos tipos cambian entre AVR y ESP32. Por eso en registros y comunicación conviene usar tipos de tamaño definido.

Tiempo: delay y millis()

delay() espera una cantidad de milisegundos y bloquea el flujo del programa mientras espera. millis() devuelve los milisegundos transcurridos desde el arranque y sirve para controlar el tiempo sin detener continuamente las demás tareas.

Ejemplo de idea: guardar un tiempo inicial y revisar si millis() - anterior >= intervalo. Así podemos revisar un pulsador mientras esperamos para cambiar un LED.

Pulsadores y resistencias

Cuando un pin queda sin nivel fijo puede estar flotante. El pull-up mantiene normalmente la entrada en 1 y al conectar el botón a GND, cuando se presiona se lee 0.

- Pull-up: botón suelto = 1; botón presionado = 0 (si va conectado a GND).
- Rebote: al pulsar pueden aparecer varios cambios rápidos en la señal. Se utiliza antirrebote para no contar varias pulsaciones.
- Flanco: cuando cambia el estado; de 0 a 1 es subida y de 1 a 0 es bajada.

No es lo mismo mantener un botón presionado que detectar el momento exacto en que cambia. Para detectar una pulsación se puede comparar la lectura actual con la anterior.

Otras anotaciones de la clase

También aparecen UART, TXD y RXD, que se utilizan para la comunicación serial; y GPIO como conexión de entrada o salida. Hay que comprobar los pines de cada placa antes de conectar los circuitos.

En resumen, en esta semana pasamos de los registros del AVR a los pines y registros del ESP32 y vimos cómo usar variables, tiempos y botones dentro de un programa que se repite.